Mini Survivors — Programming Curriculum (JavaScript)

A beginner curriculum for someone who has **never written a line of code**. You build the game together from the starter file, one small piece at a time.

- This is the **JavaScript / browser** track. There's a matching **Python** version (`Curriculum-Python.pdf`) that builds the exact same game with pygame.
- **New to code? Start with `primer.html`** — a 15-minute warm-up that prints the JavaScript building blocks the game uses (variables, arrays, `if`, loops, functions) onto the page. No game, just "edit → refresh → see."
- Then build from `starter.html` (just a blue dot + the stages marked out).
- `index.html` is the finished **answer key** — peek only when stuck.

The code is laid out in **build order: Stage 1 at the top, Stage 7 at the bottom**, so you mostly work straight down the file. Two functions at the bottom — `draw()` and `loop()` — grow as you go: each stage you uncomment one line in them. That bottom area is the only place you scroll back to.

The 5 golden rules of teaching this

1. **Run after every single change.** Even one line. "Change → run → look" is the heartbeat of the whole thing.
2. **Let her type, even when it's slower.** Her hands on the keyboard, her typos, her fixing them — that's the learning.
3. **Break it on purpose.** Change a `3` to `300`, delete a line, see what happens. Errors are how you discover what each piece was doing.
4. **One new idea per stage.** Don't stack concepts.
5. **Browser first (`starter.html`).** Zero setup, instant feedback. Save Python for the capstone.

Stage 0 — "What even is code?" (15 min)

Goal: Demystify. Code is a list of instructions the computer follows top to bottom, very fast and very literally.

Do: Open `starter.html` in a browser (you'll see a blue dot). Open the same file in a text editor. Find these two lines near the top and change the words:

```
<h1>Mini Survivors</h1>
<p>Right now this is just a blue dot. Follow the stages to build the game.</p>
```

Save, refresh the browser. **The screen changed because you changed the file.**

The aha: *I edit the file, I refresh, the world changes. That's programming.*

Stage 1 — Variables & drawing a dot (already done for you)

New concept: Variables (named boxes holding a value) and the screen as an x/y grid. Top-left is (0,0); x goes right, y goes **down**.

The starter already contains Stage 1 — read it together so it isn't magic. It's three things: the **player** (a bag of numbers), a **circle** helper, and **draw()** which puts the player on screen:

```
// the player is just a bag of numbers
let player = { x: W / 2, y: H / 2, size: 12, speed: 3 };

// a helper that draws one filled circle
function circle(x, y, size, color) {
  ctx.fillStyle = color;
  ctx.beginPath();
  ctx.arc(x, y, size, 0, Math.PI * 2);
  ctx.fill();
}

// draw the world (just the player for now)
function draw() {
  ctx.clearRect(0, 0, W, H);                  // wipe the screen
  circle(player.x, player.y, player.size, "#5af");
}
```

Try it: make the dot bigger, move it to a corner, change its color `"#5af"`.

The aha: *A "thing" on screen is just a few numbers I picked.*

Stage 2 — The game loop & moving (45 min)

New concept: The **game loop** — `update → draw`, ~60×/second, forever. Movement is just "change x a little, every frame." The `loop()` at the bottom of the file already runs; you'll start adding to it now.

In the Stage 2 area, add the keyboard input and `movePlayer()`:

```
// remember which keys are held down
let keys = {};
document.addEventListener("keydown", e => keys[e.key.toLowerCase()] = true);
document.addEventListener("keyup",   e => keys[e.key.toLowerCase()] = false);

// move the player based on the keys
function movePlayer() {
  if (keys["w"] || keys["arrowup"])    player.y -= player.speed;
  if (keys["s"] || keys["arrowdown"])  player.y += player.speed;
  if (keys["a"] || keys["arrowleft"])  player.x -= player.speed;
  if (keys["d"] || keys["arrowright"]) player.x += player.speed;

  // keep the player inside the screen
  player.x = Math.max(player.size, Math.min(W - player.size, player.x));
  player.y = Math.max(player.size, Math.min(H - player.size, player.y));
}
```

↩ **In `loop()` (bottom), uncomment:**

```
movePlayer();    // Stage 2
```

Try it: change `speed`. Make a key teleport you to the middle.

The aha: *Motion is just a number changing a tiny bit, many times a second.*

Stage 3 — Lists & spawning enemies (45 min)

New concept: Lists/arrays (hold *many* things) and a little randomness.

In the Stage 3 area, add the enemies list + frame counter, and the spawner:

```
// a list to hold all enemies, and a clock
let enemies = [];
let frame = 0;

// every so often, add an enemy at a random edge
function spawnEnemy() {
  const spawnEvery = Math.max(6, 45 - frame / 120);   // speeds up over time
  if (frame % Math.floor(spawnEvery) !== 0) return;

  let x, y;
  if (Math.random() < 0.5) {                  // left or right edge
    x = Math.random() < 0.5 ? 0 : W;
    y = Math.random() * H;
  } else {                                    // top or bottom edge
    x = Math.random() * W;
    y = Math.random() < 0.5 ? 0 : H;
  }
  enemies.push({ x: x, y: y, size: 10, speed: 1.6 });
}
```

↩ **In `draw()`, uncomment** the enemies line, and **in `loop()`, uncomment** the two Stage-3 lines:

```
for (let e of enemies) circle(e.x, e.y, e.size, "#e55");   // Stage 3: enemies
```

```
frame++;          // Stage 3
spawnEnemy();     // Stage 3
```

Try it: spawn faster/slower (change the numbers), make enemies bigger.

The aha: *I don't make 50 enemy variables. I make one list of 50 things.*

Stage 4 — Loops & chasing (45 min)

New concept: `for` loops — "do the same thing to every item in a list." Also a first helper function: **distance** between two points.

In the Stage 4 area, add the distance helper and enemy movement:

```
// how far apart are two points? (straight from math class)
function distance(ax, ay, bx, by) {
  const dx = ax - bx;
  const dy = ay - by;
  return Math.sqrt(dx * dx + dy * dy);
}

// move every enemy one step toward the player
function moveEnemies() {
  for (let e of enemies) {
    const d = distance(e.x, e.y, player.x, player.y);
    if (d === 0) continue;                    // never divide by zero
    e.x += ((player.x - e.x) / d) * e.speed;   // this line points the enemy at you
    e.y += ((player.y - e.y) / d) * e.speed;
  }
}
```

↩ **In `loop()`, uncomment:**

```
moveEnemies();   // Stage 4
```

Try it: change enemy `speed`. Flip a `+` to a `-` so they run *away* (funny, and instructive).

The aha: *One loop handles 1 enemy or 1,000. I wrote the rule once.*

Stage 5 — Functions & auto-shooting (45 min)

New concept: Functions — a named box of instructions, so the program reads like a to-do list. (You've been writing them already; now name the idea.)

First, ↩ **go back to the Stage 1 player** and add a fire rate:

```
fireRate: 30,    // shoot every 30 frames. Lower = faster.
```

In the Stage 5 area, add the bullets list and the shooter:

```
// a list to hold bullets
let bullets = [];

// every fireRate frames, fire a bullet at the nearest enemy
function autoShoot() {
  if (frame % player.fireRate !== 0) return;   // not time yet
  if (enemies.length === 0) return;            // nothing to shoot

  // find the nearest enemy
  let target = enemies[0];
  let best = Infinity;
  for (let e of enemies) {
    const d = distance(player.x, player.y, e.x, e.y);
    if (d < best) { best = d; target = e; }
  }

  // fire a bullet aimed at it
  const d = distance(player.x, player.y, target.x, target.y);
  bullets.push({
    x: player.x, y: player.y,
    dx: ((target.x - player.x) / d) * 6,       // 6 = bullet speed
    dy: ((target.y - player.y) / d) * 6,
    size: 4,
  });
}
```

↩ **In `draw()`, uncomment** the bullets line, and **in `loop()`, uncomment** `autoShoot()`:

```
for (let b of bullets) circle(b.x, b.y, b.size, "#ff5");   // Stage 5: bullets
```

```
autoShoot();     // Stage 5
```

(Bullets won't move yet — that's the next stage.)

Try it: shoot faster (`fireRate`), bigger bullets, fire two at once.

The aha: *The loop reads like a to-do list: move, spawn, chase, shoot.*

Stage 6 — Conditionals & collision (45 min)

New concept: `if` statements (decisions) and **collision** — two circles touch when the distance between them is less than their sizes added up.

In the Stage 6 area, add the score + game-over flag, and `moveBullets()`:

```
// score, and a flag for whether the game has ended
let score = 0;
let gameOver = false;

// move bullets, delete off-screen ones, kill enemies on contact
function moveBullets() {
  for (let i = bullets.length - 1; i >= 0; i--) {   // backwards = safe to remove
    const b = bullets[i];
    b.x += b.dx;
    b.y += b.dy;

    if (b.x < 0 || b.x > W || b.y < 0 || b.y > H) { // off screen?
      bullets.splice(i, 1);
      continue;
    }

    for (let j = enemies.length - 1; j >= 0; j--) {
      const e = enemies[j];
      if (distance(b.x, b.y, e.x, e.y) < b.size + e.size) {   // touching?
        score += 10;
        bullets.splice(i, 1);    // remove the bullet
        enemies.splice(j, 1);    // remove the enemy
        break;
      }
    }
  }
}
```

↩ **Back up in `moveEnemies()`**, add one line inside the `for` loop so touching the player ends the game:

```
if (d < e.size + player.size) gameOver = true;
```

↩ **In `loop()`, uncomment:**

```
moveBullets();   // Stage 6
```

> Nothing happens *visually* when `gameOver` becomes true yet — the next stage adds the game-over screen. For now, watch your bullets delete enemies and the (hidden) score climb.

Try it: make the player's hitbox tiny (hard) or huge (easy). Make enemies take 2 hits (give them `hp: 2`, subtract 1 per hit — you're reinventing a feature!).

The aha: *The whole game is just "if this is true, do that," repeated.*

Stage 7 — The clock, HUD, game-over screen & restart (45 min)

New concept: showing the game's **state** on screen, and **restarting**. This is the stage where you *wrap* the loop — the one time we restructure it.

In the Stage 7 area, add a survival clock and the restart:

```
let startTime = performance.now();   // when this run began (real time, in ms)
let seconds = 0;                     // seconds survived (freezes when you die)

function resetGame() {
  enemies = [];
  bullets = [];
  frame = 0;
  score = 0;
  gameOver = false;
  player.x = W / 2;
  player.y = H / 2;
  startTime = performance.now();
  seconds = 0;
}

// Press R to play again, once you're dead.
document.addEventListener("keydown", e => {
  if (e.key.toLowerCase() === "r" && gameOver) resetGame();
});
```

In `draw()`, uncomment / add the HUD at the bottom:

```
ctx.fillStyle = "#fff";
ctx.font = "14px monospace";
ctx.textAlign = "left";
ctx.fillText("Time: " + seconds + "s", 10, 20);
ctx.fillText("Score: " + score, 10, 40);
```

Add a **game-over screen** function (next to `draw()`):

```
function drawGameOver() {
  ctx.fillStyle = "rgba(0,0,0,0.6)";
  ctx.fillRect(0, 0, W, H);
  ctx.fillStyle = "#fff";
  ctx.textAlign = "center";
  ctx.font = "30px monospace";
  ctx.fillText("GAME OVER", W / 2, H / 2 - 10);
  ctx.font = "16px monospace";
  ctx.fillText("You survived " + seconds + " seconds", W / 2, H / 2 + 20);
  ctx.fillText("Final score: " + score, W / 2, H / 2 + 45);
  ctx.fillText("Press R to play again", W / 2, H / 2 + 75);
}
```

Finally, **wrap the loop** so it shows that screen when you're dead, and counts your survival time while alive:

```
function loop() {
  if (gameOver) {
    drawGameOver();
  } else {
    movePlayer();    // Stage 2
    frame++;         // Stage 3
    spawnEnemy();    // Stage 3
    moveEnemies();   // Stage 4
    autoShoot();     // Stage 5
    moveBullets();   // Stage 6
    seconds = Math.floor((performance.now() - startTime) / 1000);
    draw();
  }
  requestAnimationFrame(loop);
}
loop();
```

That's the whole game. 🎉

Try it: worth more points per kill; add `score += 1` each frame to reward survival; show a high score (intro to *saving data* with `localStorage`).

The aha: *Most "features" are the same recipe: make a value, change it, show it.*

Capstone — "Same game, another language" (30 min)

Open `game.py` next to `index.html`. They share the same **Stage** labels. Skim them together.

The ideas — variables, loops, lists, functions, ifs — are **identical**. Only the spelling changes:

Idea	JavaScript	Python
Add to a list	`enemies.push(x)`	`enemies.append(x)`
Loop a list	`for (let e of enemies)`	`for e in enemies:`
Block of code	`{ ... }`	`:` + indentation
The game loop	`requestAnimationFrame(loop)`	`while running:`
Square root	`Math.sqrt(...)`	`math.hypot(...)`

The point to land: *you didn't learn JavaScript — you learned to program,* and that transfers to any language.

> Python needs a one-time `pip install pygame` (or the included `venv`). The browser needs nothing — that's why we started there.

Where to go next (pick what excites her)

- **More weapons:** spread shot, a slow huge bullet, an orbiting shield.
- **Enemy variety:** fast-weak vs slow-tanky (bring back `hp`).
- **Power-ups:** the green XP gems again — but as a thing *she* designs.
- **Juice:** screen shake, particles on kill, sound. Small effort, huge payoff.
- **Save the high score** (`localStorage`) — first taste of data that outlives the program.

The goal was never to finish a game. It's to make her believe **she can make the computer do what she wants.** Once that clicks, she'll lead.